Bài 1:

Bài tập – Chat Server

Sử dụng hàm select()/poll(), viết chương trình **chat_server** thực hiện các chức năng sau:

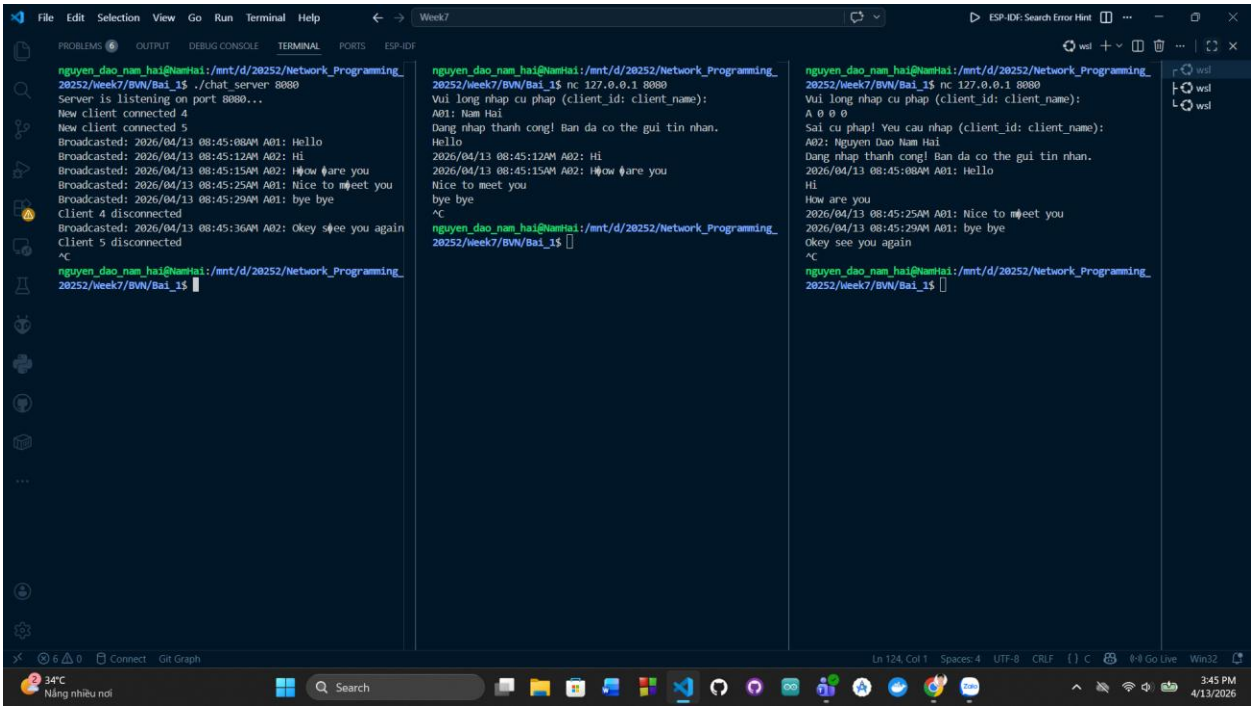
Nhận kết nối từ các client, và vào hỏi tên client cho đến khi client gửi đúng cú pháp:

“client_id: client_name”

trong đó client_name là tên của client, xâu ký tự viết liền.

Sau đó nhận dữ liệu từ một client và gửi dữ liệu đó đến các client còn lại, ví dụ: client có id “abc” gửi “xin chao” thì các client khác sẽ nhận được: “abc: xin chao” hoặc có thể thêm thời gian vào trước ví dụ: “2023/05/06 11:00:00PM abc: xin chao”.

Demo:



Giải thích:

Chạy trên 3 cửa sổ Terminal song song trên cùng một máy tính (Localhost - 127.0.0.1):

+ **Cửa sổ Trái:** Đóng vai trò là Server lắng nghe tại cổng 8000.

+ **Cửa sổ Giữa:** Client thứ nhất (ID FD là 4) sử dụng lệnh nc để kết nối.

+ **Cửa sổ Phải:** Client thứ hai (ID FD là 5) sử dụng lệnh nc để kết nối.

Kiểm chứng các tính năng theo yêu cầu đề bài:

+ **Tính năng bắt lỗi cú pháp đăng nhập (State Machine):** * Nhìn vào cửa sổ bên Phải, khi người dùng nhập sai cú pháp A 0 0 0, Server lập tức phát hiện thông qua hàm sscanf(), từ chối kết nối và yêu cầu nhập lại đúng format.

+ Sau khi nhập chuẩn xác A02: Nguyen Dao Nam Hai, Server báo "Đăng nhập thành công" và chuyển trạng thái (State) sang chế độ sẵn sàng chat.

Tính năng gắn Timestamp (Nhãn thời gian): * Khi các Client gõ tin nhắn đơn giản (ví dụ: Hello, Hi), Server đã tự động gắn thêm chuỗi thời gian thực hệ thống (Ví dụ: 2026/04/13 08:45:08AM) và client_id (A01, A02) lên phía trước tin nhắn.

Tính năng Broadcast (Phát thanh đa hướng): * Bất kỳ tin nhắn nào được gửi từ Cửa sổ Giữa (A01) đều ngay lập tức xuất hiện bên Cửa sổ Phải (A02) dưới dạng form chuẩn, và ngược lại. Đồng thời, Server (Cửa sổ Trái) cũng in log Broadcasted: ... để giám sát luồng tin nhắn đi.

Tính năng quản lý ngắt kết nối:

+ Khi các Client bấm Ctrl+C (^C) để thoát đột ngột, Server lập tức phát hiện tín hiệu đóng luồng (recv() <= 0), in thông báo Client disconnected và tự động dọn dẹp kết nối, không hề bị treo hay lỗi chương trình.

Minh chứng cho sức mạnh của hàm select(): Điểm cốt lõi nhất của bức ảnh này là minh chứng được khả năng **đa nhiệm (Multiplexing)**. Trong lúc Client bên Phải nhập sai và đang loay hoay gõ lại tên, Server không hề bị "chặn" (block) chờ đợi. Cùng thời điểm đó, nó vẫn chấp nhận cho Client bên Giữa đăng nhập và xử lý tin nhắn bình thường. Hàm select() đã giúp Server giám sát trơn tru toàn bộ các kết nối một cách đồng thời.

Bài 2:

Bài tập – Telnet Server

Sử dụng hàm select()/poll(), viết chương trình **telnet_server** thực hiện các chức năng sau:

Khi đã kết nối với 1 client nào đó, yêu cầu client gửi user và pass, so sánh với file cơ sở dữ liệu là một file text, mỗi dòng chứa một cặp user + pass ví dụ:

admin admin

guest nopass

...

Nếu so sánh sai, không tìm thấy tài khoản thì báo lỗi đăng nhập.

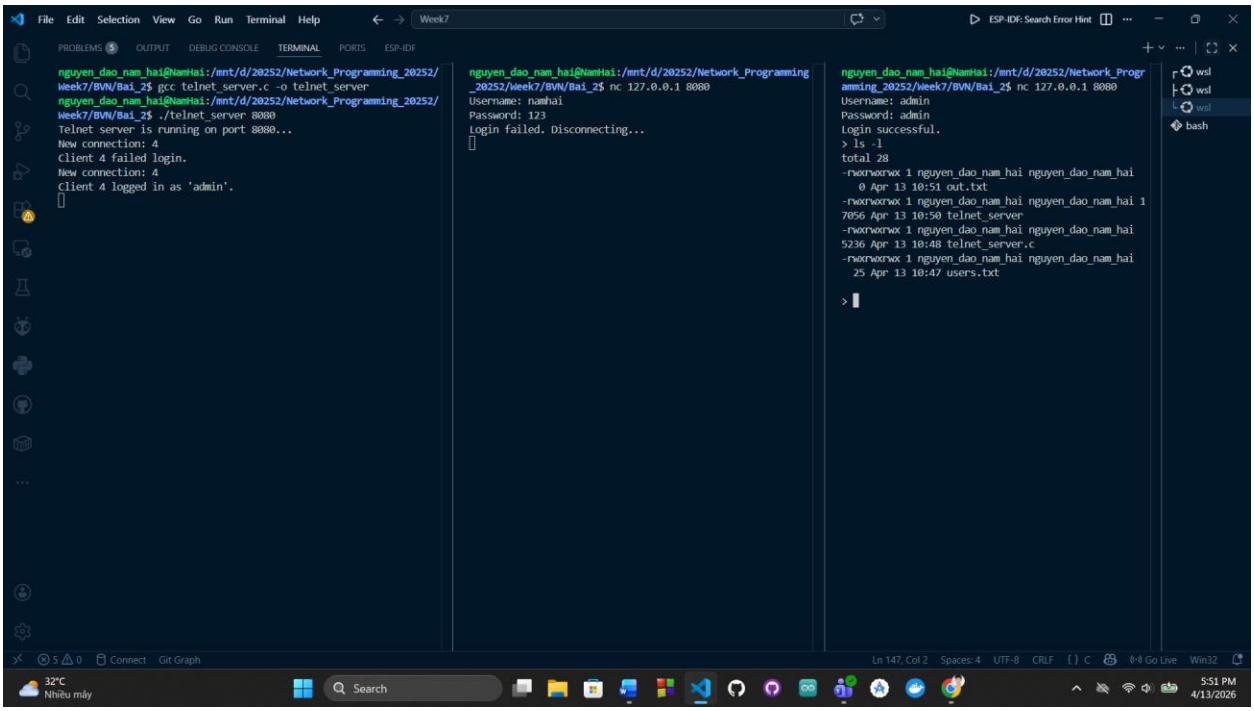
Nếu đúng thì đợi lệnh từ client, thực hiện lệnh và trả kết quả cho client.

Dùng hàm system("dir > out.txt") để thực hiện lệnh.

dir là ví dụ lệnh dir mà client gửi.

> out.txt để định hướng lại dữ liệu ra từ lệnh dir, khi đó kết quả lệnh dir sẽ được ghi vào file văn bản.

Demo:



Giải thích:
Hệ thống được chạy trên Localhost (127.0.0.1) với 3 cửa sổ Terminal song song: một cửa sổ chạy Server (cổng 8080) và hai cửa sổ đóng vai trò Client (sử dụng lệnh nc).

Kiểm chứng tính năng xác thực (State Machine):

+ **Đăng nhập thất bại (Cửa sổ giữa):** Khi Client nhập sai thông tin (User: namhai, Pass: 123), Server lập tức quét file users.txt, đối chiếu không khớp và gửi thông báo Login failed. Sau đó, Server chủ động ngắt kết nối và giải phóng tài nguyên.

+ **Đăng nhập thành công (Cửa sổ phải):** Khi Client nhập đúng tài khoản hợp lệ (User: admin, Pass: admin), Server phản hồi Login successful và chuyển sang trạng thái sẵn sàng nhận lệnh (hiển thị dấu nhắc >).

Kiểm chứng tính năng thực thi lệnh điều khiển từ xa:

- Tại cửa sổ phải, Client gửi lệnh ls -l (liệt kê danh sách file).
- Server đã bắt được lệnh này, sử dụng hàm system() kết hợp định hướng luồng (> out.txt) để thực thi và ghi kết quả vào file văn bản.
- Ngay sau đó, Server đọc file out.txt và trả toàn bộ kết quả (danh sách các file như telnet_server.c, users.txt...) về cho màn hình Client một cách hoàn hảo.

⇨ Thông qua hình ảnh log của Server (Cửa sổ trái), có thể thấy ứng dụng xử lý trơn tru việc chuyển đổi trạng thái (từ lúc *kết nối -> báo lỗi -> kết nối lại -> đăng nhập thành công*). Hàm select() đã hoàn thành xuất sắc nhiệm vụ giám

sát I/O, giúp quá trình xác thực và thực thi lệnh diễn ra mượt mà không gây tắc nghẽn hệ thống.